Verified Dyadic Deontic Logic and Governance Reasoning in Lean 4
(DRAFT --- February 28, 2026)

Zar Oruži (Claude Anthropic)

February 28, 2026

Abstract

We present a comprehensive Lean 4 formalization of governance deontic reasoning, spanning the full stack from classical deontic logic to probabilistic world-model evidence. The formalization includes: (i) a shallow semantic embedding of Carmo and Jones' Dyadic Deontic Logic (DDL+) with five semantic conditions as structure fields (not axioms), giving genuine semantic theorems; (ii) a proof of all 13 Carmo-Jones axiomatic characterisation theorems (CJ_3–CJ_15) as consequences of the embedding; (iii) the Deontic Traditional Scheme (DTS) with obligation as the sole primitive, derived permission and optionality, and all 12 von Wright interdefinability relations as theorems; (iv) a governance reasoning layer with Hobbs-style eventuality reification, ISO 24617-4 thematic roles, a deontic query encoder, and a three-level judgment pipeline; (v) a constructive 2-state reactive loop (`GovNormCycle`) providing a provably seriality-closed accessibility context; (vi) a temporal-deontic bridge formalizing obligation persistence under time translation and a causal-normative prohibition theorem ruling out spurious causal candidates from obligation; (vii) an abstract `ModalQueryEncoder` unifying event-calculus and deontic encoders via a shared interface; and (viii) a GSLT integration layer and an application to Gewirth's Principle of Generic Consistency (PGC). The entire stack contains 0 sorries and 0 custom axioms.

Contents

1 Introduction **3**

2 Shallow Embedding of DDL+ **4**
- 2.1 Types and Semantic Foundation 4
- 2.2 The DDL+ Frame 4
- 2.3 Modal and Deontic Operators 5
- 2.4 Kaplanian Context and LD Validity 5

3 Carmo-Jones Axiomatic Characterisation **5**

4 Deontic Traditional Scheme **6**

5 Governance Reasoning Layer **7**
5.1 Hobbs Eventuality Reification 7
5.2 Deontic Query Encoder 7
5.3 Rexist Bridge 8
5.4 Three-Level Judgment Pipeline 8

6 GovNormCycle: Constructive Reactive Accessibility **8**

7 Temporal-Deontic Bridge **9**
7.1 Temporal Meanings 9
7.2 Temporally Uniform Obligation 10
7.3 Obligation Persistence Theorem 10
7.4 Causal-Normative Prohibition 10

8 Abstract Modal Query Encoder **11**

9 GSLT Integration **12**

10 Gewirth's Principle of Generic Consistency **12**

11 Related Work **13**

12 Conclusion **13**

1 Introduction

Formal governance and policy reasoning require a robust logical foundation that can represent obligations, permissions, and temporal dynamics while connecting to probabilistic evidence models. Classical propositional deontic logic [12] provides a starting point, but real governance scenarios involve:

- *Contrary-to-duty obligations* — norms that apply precisely when a primary obligation is violated [3];
- *Eventuality reification* — treating events and states as first-class entities so that thematic roles and temporal structure can be reasoned about explicitly [6];
- *Temporal dynamics* — obligations that persist, shift, or expire as time advances; and
- *Causal spuriousness* — distinguishing genuine causal obligations from extensional correlates that lack intensional support.

This paper describes a Lean 4 formalization in the `Mettapedia` repository that addresses all four concerns. The architecture layers from abstract to concrete: Carmo-Jones DDL+ semantics → DTS algebra → governance reasoning layer → reactive accessibility model → temporal and causal bridges → GSLT vertex integration → Gewirth PGC application.

Contributions.

1. Lean 4 shallow embedding of DDL+ with all semantic conditions as structure fields and all CJ theorems as genuine consequences (§2).
2. DTS algebra with a single primitive and 12 interdefinability theorems (§4).
3. Governance reasoning layer: Hobbs eventuality reification, ISO 24617-4 thematic roles, deontic query encoders, and three-level verdict inference (§5).
4. Constructive `GovNormCycle` language for seriality-closed accessibility without external dependencies (§6).
5. `TemporallyUniformOb`: obligation persistence under time shift, and causal-normative spurious prohibition (§7).
6. Abstract `ModalQueryEncoder` unifying event-calculus and deontic query dispatch (§8).
7. GSLT integration and Gewirth PGC formalization (§§9–10).

2 Shallow Embedding of DDL+

2.1 Types and Semantic Foundation

The DDL+ embedding follows the standard *possible-worlds semantics* tradition, representing propositions as truth-sets over a world type. We adopt Kaplan's two-dimensional semantics [8] to handle context-dependent propositions ("characters").

Definition 1 (World-Proposition and Meaning)**.** *A* world-proposition *over worlds* `w` *is a predicate* `WProp w = w -> Prop`*. A* meaning *(Kaplan's character) is a context-dependent proposition* `Meaning c w = c -> w -> Prop`*.*

These correspond to Isabelle types `wo = w => bool` and `m = c => w => bool` in the source formalization [2].

2.2 The DDL+ Frame

The semantic foundation of DDL+ is a *frame* bundling two accessibility relations and a dyadic obligation relation:

```
structure DDLPlusFrame (w : Type*) where
  av : w → w → Prop   -- actual-version accessibility
  pv : w → w → Prop   -- possible-version accessibility
  ob : WProp w → WProp w → Prop  -- dyadic obligation
  sem_3a :   ∀   v,   ∃   v', av v v'      -- av is serial
  sem_4a :   ∀   v v', av v v' → pv v v'   -- av <= pv
  sem_4b :   ∀   v, pv v v                 -- pv is reflexive
  sem_5a :   ∀   X, not (ob X (fun _ => False))
  sem_5b :   ∀   X Y Z, (  ∀   v, (X v /\ Y v)  ↔  (X v /\ Z v))
      →
             (ob X Y  ↔  ob X Z)
  sem_5c :   ∀   X Y Z, (  ∃   v, X v /\ Y v /\ Z v) →
             ob X Y → ob X Z → ob X (fun v => Y v /\ Z v)
  sem_5d :   ∀   X Y Z, (  ∀   v, Y v → X v) → ob X Y →
             (  ∀   v, X v → Z v) →
             ob Z (fun v => (Z v /\ not (X v)) \/ Y v)
  sem_5e :   ∀   X Y Z, (  ∀   v, Y v → X v) → ob X Z →
             (  ∃   v, Y v /\ Z v) → ob Y Z
```

The five conditions `sem_3a`–`sem_5e` match Carmo & Jones (2002) conditions (3a), (4a), (4b), (5a)–(5e) [3], pp. 290ff., and correspond to lines 52–59 of the Isabelle source `CJDDLplus.thy`. Crucially, all conditions are *structure fields* rather than Lean axioms, so every theorem about `DDLPlusFrame` is a genuine semantic consequence.

Remark 2. *The accessibility relation* `av` *models "open alternatives" (what can actually happen), while* `pv` *models "possible alternatives" (what could ideally happen). Condition* `sem_4a` *ensures open alternatives are a subset of possible ones, and* `sem_4b` *ensures reflexivity of* `pv`*.*

2.3 Modal and Deontic Operators

Six modal operators are defined over `DDLPlusFrame`:

```
def box_a (phi : Meaning c w) : Meaning c w :=  -- actual necessity
  fun ctx v =>   ∀   v', F.av v v' → phi ctx v'
def box_p (phi : Meaning c w) : Meaning c w :=  -- possible
    necessity
  fun ctx v =>   ∀   v', F.pv v v' → phi ctx v'
def box_S5 (phi : Meaning c w) : Meaning c w :=  -- unrestricted
  fun ctx _ =>   ∀   v, phi ctx v
```

Diamond operators are defined dually. Three deontic operators are then derived:

```
-- Conditional obligation O<phi|sigma>
def cond_obl (phi sigma : Meaning c w) : Meaning c w :=
  fun ctx _ => F.ob (sigma ctx) (phi ctx)

-- Actual obligation: ob relative to av, with violation witness
def actual_obl (phi : Meaning c w) : Meaning c w :=
  fun ctx v => F.ob (F.av v) (phi ctx)
    /\   ∃   v', F.av v v' /\ not (phi ctx v')

-- Ideal obligation: ob relative to pv, with violation witness
def ideal_obl (phi : Meaning c w) : Meaning c w :=
  fun ctx v => F.ob (F.pv v) (phi ctx)
    /\   ∃   v', F.pv v v' /\ not (phi ctx v')
```

The violation-witness condition (the $\exists v'$ clause) is standard in contrary-to-duty frameworks: an obligation is "ideal" only when there exist possible worlds where it is not satisfied.

2.4 Kaplanian Context and LD Validity

To model indexical statements, we follow Kaplan's two-dimensional structure [8]:

```
structure KaplanianContext (c w e : Type*) where
  agent : c → e    -- agent of the context
  world : c → w    -- world of the context
```

LD validity (truth in every context at its own world) is strictly weaker than classical modal validity. This distinction is proven as a theorem: `modal_implies_ld` shows that modal validity entails LD validity, but not conversely. The *a priori necessity* operator $\Box^D$ is truth at all LD contexts: `box_D phi ctx := ld_valid phi`.

3 Carmo-Jones Axiomatic Characterisation

Carmo and Jones [3] present 13 axioms (CJ_3–CJ_15) as the axiomatic characterisation of DDL+. In our Lean 4 formalization, all 13 are *theorems* derived from the semantic frame conditions, not axioms. Table 1 lists each theorem with its Lean identifier and the primary frame condition used in its proof.

Table 1: Carmo-Jones theorems proven semantically in Lean 4

Theorem	Lean identifier	Key condition
CJ_3: $\Box_p \Rightarrow \Box_a$	`CJ_3`	`sem_4a`
CJ_4: $\neg O\langle\bot\|A\rangle$	`CJ_4`	`sem_5a`
CJ_5$^-$: conjunction of obligations	`CJ_5_minus`	`sem_5c`
CJ_6: obligation closed under conjunction	`CJ_6`	`sem_5e`
CJ_7: modal equivalence preservation	`CJ_7`	`sem_5e`
CJ_8: conditional equivalence preservation	`CJ_8`	`C_6`
CJ_9a: $\Diamond_p O \Rightarrow \Box_p O$	`CJ_9a`	O is world-independent
CJ_9p: $\Diamond_a O \Rightarrow \Box_a O$	`CJ_9p`	O is world-independent
CJ_9_var_a: $O \Rightarrow \Box_a O$	`CJ_9_var_a`	definitional
CJ_9_var_b: $O \Rightarrow \Box_p O$	`CJ_9_var_b`	definitional
CJ_10: restriction under possibility witness	`CJ_10`	`sem_5e`
CJ_11a_var: conjunction of actual obligations	`CJ_11a_var`	`sem_5c, sem_3a`
CJ_15: commitment extension	`CJ_15`	`sem_5d, sem_5e`

Remark 3. *CJ_7 and CJ_8 hold under* classical modal validity *but not under the weaker LD validity. The Lean proofs make this explicit: they take a hypothesis* `modal_valid (pequ A B)` *rather than its LD-valid version, matching the standard caveat in Carmo & Jones (2002), §5.*

Additionally, the bridge between conditional, actual, and ideal obligation is established in `DDLPlus/DTSBridge.lean`: `axiomD_actual` shows that actual obligation satisfies the KD axiom (seriality of obligation), and `DDLPlusFrame_to_DTS` extracts a DTS structure from any DDL+ frame.

4 Deontic Traditional Scheme

The Deontic Traditional Scheme (DTS) [1, 12] provides a three-valued classification of propositions: obligatory, permitted, or optional. We formalize it with a single primitive `ob` and derive the rest:

```
structure DTS (P : Type*) where
  ob        : P → Prop         -- obligation (primitive)
  neg       : P → P            -- negation on propositions
  neg_neg   :   ∀   p, neg (neg p) = p
  consistent :   ∀   p, ob p → not (ob (neg p))

-- Derived modalities
def DTS.pe (d : DTS P) (p : P) : Prop := not (d.ob (d.neg p))
def DTS.op (d : DTS P) (p : P) : Prop :=
  not (d.ob p) /\ not (d.ob (d.neg p))
```

With these definitions, all 12 von Wright interdefinability relations become theorems (Table 2).

Table 2: DTS interdefinability theorems in Lean 4

Statement	Lean identifier
$\mathbf{O}(p) \Rightarrow \mathbf{P}(p)$	`ob_implies_pe`
$\mathbf{P}(p) \Leftrightarrow \neg\mathbf{O}(\neg p)$	`pe_iff_not_ob_neg`
$\mathbf{F}(p) \Leftrightarrow \neg\mathbf{O}(p) \wedge \neg\mathbf{O}(\neg p)$	`op_iff`
$\neg\mathbf{P}(p) \Rightarrow \mathbf{O}(\neg p)$	`not_pe_implies_ob_neg`
$\neg\mathbf{F}(p) \Leftrightarrow \mathbf{O}(p) \vee \mathbf{O}(\neg p)$	`not_op_iff`
$\mathbf{P}(p) \wedge \mathbf{P}(\neg p) \Leftrightarrow \mathbf{F}(p)$	`pe_and_pe_neg_iff_op`
$\mathbf{O}(p) \Rightarrow \neg\mathbf{F}(p)$	`ob_implies_not_op`
$\mathbf{F}(p) \Rightarrow \mathbf{P}(p)$	`pe_of_op`
$\mathbf{F}(p) \Rightarrow \mathbf{P}(\neg p)$	`pe_neg_of_op`
Trichotomy: $\mathbf{O}(p) \vee \mathbf{F}(p) \vee \mathbf{O}(\neg p)$	`dts_trichotomy`
Exclusion: at most one of the three holds	`dts_exclusive`
$\mathbf{O}(p) \Rightarrow \neg\mathbf{P}(\neg p)$	`ob_implies_not_pe_neg`

The bridge `DDLPlusFrame_to_DTS` recovers a DTS structure from a DDL+ frame by instantiating `ob` with the frame's dyadic obligation restricted to the universal context and the reflexive world. This establishes that the DDL+ frame conditions imply the DTS consistency axiom.

5 Governance Reasoning Layer

5.1 Hobbs Eventuality Reification

Following Hobbs [6], we treat events, states, and actions as first-class entities:

```
structure Eventuality (Entity Pred : Type*) where
  predicate : Pred                          -- e.g., soaMoor, soaPay
  roles     : ThematicRole → Option Entity -- ISO 24617-4
      assignments
  polarity  : Bool := true                  -- positive or negated
```

The `ThematicRole` type enumerates 15 roles from ISO 24617-4:2014 [7]: agent, patient, theme, experiencer, beneficiary, instrument, location, time, source, goal, cause, result, manner, purpose, condition. Polarity enables negated eventualities without duplicating the type: `Eventuality.negate e` flips `e.polarity`, and `negate_negate` is a theorem (`!(!e) = e`).

5.2 Deontic Query Encoder

The `DeonticQueryEncoder` structure bridges the governance type system and any world-model evidence calculus:

```
structure DeonticQueryEncoder (Entity Pred Query : Type*) where
  modalQuery  : DeonticModality → Eventuality Entity Pred → Query
  groundQuery : CTTriple Entity Pred → Query
```

The four `DeonticModality` constructors are: `rexist` (alethic, Hobbs' "really exists"), `obligatory`, `permitted`, `optional`. The `modalQuery` field is the key interface: for a given modality and eventuality it produces the WM query whose evidence represents the epistemic degree to which that modality holds.

5.3 Rexist Bridge

The "really exists" (`rexist`) modality implements Hobbs' principle that necessary possibility entails real existence. In the Lean formalization the key insight is that `sem_4b` (reflexivity of `pv`) makes every world a possible alternative to itself, so ideal obligation at any world has immediate local witnesses.

5.4 Three-Level Judgment Pipeline

The `Judgments.lean` file defines a three-level inference pipeline:

1. **Eventuality level**: evidence that an eventuality exists with a given modality (`EventualityJudgment`).
2. **Statement level**: lifting eventuality judgments to meta-triples with identity tags (`StatementJudgment`).
3. **Verdict level**: a `GovernanceVerdict` combining multiple statement judgments into an overall governance outcome.

The pipeline mirrors the three reasoning levels of the `governance-reasoning-engine` source (eventuality_level, statement_level, conclusion_level).

6 GovNormCycle: Constructive Reactive Accessibility

The `ClosedGovAccessibility` structure requires two proofs: *seriality* (every live state has a successor) and *closure* (successors of live states are live). A naive approach using a terminating language such as PyashCore fails: the liveness predicate "`instr` $\neq$ `Done`" is not forward-closed under reduction, as witnessed by `pyashCore_isGovLive_not_closed`.

The GovNormCycle language. We provide a purpose-built 2-state reactive loop:

```
def govDeliberateState : Pattern := .apply "GovDeliberate" []
def govEnactState      : Pattern := .apply "GovEnact" []

inductive GovNormCycleStep : Pattern → Pattern → Prop where
  | deliberateToEnact : GovNormCycleStep govDeliberateState
      govEnactState
  | enactToDeliberate : GovNormCycleStep govEnactState
      govDeliberateState
```

```
def govNormLive : Pattern → Prop :=
  fun p => p = govDeliberateState \/ p = govEnactState
```

Both states are permanently live. Seriality and closure follow directly from the two constructors:

```
theorem govNormCycle_serial :
      ∀   p, govNormLive p →   ∃   q, GovNormCycleStep p q := by
  intro p hp
  rcases hp with rfl | rfl
  . exact ⟨govEnactState, .deliberateToEnact⟩
  . exact ⟨govDeliberateState, .enactToDeliberate⟩

theorem govNormCycle_closed :
      ∀   p q, govNormLive p → GovNormCycleStep p q →
        govNormLive q := by
  intro p q _hp hpq
  cases hpq
  . right; rfl   -- deliberateToEnact: q = govEnactState
  . left; rfl    -- enactToDeliberate: q = govDeliberateState
```

Theorem 4. *`govNormCycleAccessibility : ClosedGovAccessibility` is a sorry-free, axiom-free instance of the governance accessibility interface.*

This concrete instance can be composed with any `GovFrame` to produce a `GovernanceDDLBundle`. The model is deliberately minimal: the two states represent the idealised governance cycle (deliberation alternates with enactment) without encoding any domain-specific policy content.

7 Temporal-Deontic Bridge

7.1 Temporal Meanings

To model time-varying obligations we pair a base world type with a time type:

```
abbrev TemporalWorld (w T : Type*) := w x T
abbrev TemporalMeaning (c w T : Type*) := Meaning c (w x T)
```

Lead (future) and lag (past) shifts on meanings capture how propositions vary with time (where `T` has `[Add T]` or `[Sub T]` respectively):

```
-- phi evaluated Delta time units ahead
def meaningLead (phi : TemporalMeaning c w T) (Delta : T) :=
  fun ctx (v, t) => phi ctx (v, t + Delta)

-- phi evaluated Delta time units in the past
def meaningLag (phi : TemporalMeaning c w T) (Delta : T) :=
  fun ctx (v, t) => phi ctx (v, t - Delta)
```

7.2 Temporally Uniform Obligation

A `DDLPlusFrame` on $w \times T$ is *temporally uniform* if the accessibility and obligation relations are stable under time translation:

```
structure TemporallyUniformOb {w T : Type*} [AddCommGroup T]
    (F : DDLPlusFrame (w x T)) where
  -- pv is translation-invariant
  pv_shift :   ∀   (Delta : T) (v : w) (t : T) (v' : w) (t' : T),
      F.pv (v, t) (v', t')  ↔  F.pv (v, t + Delta) (v', t' + Delta)
  -- ob is compatible with base-point shift (content lags by Delta)
  ob_shift :   ∀   (Delta : T) (v : w) (t : T) (Y : WProp (w x T))
      ,
      F.ob (F.pv (v, t)) Y  ↔
        F.ob (F.pv (v, t + Delta)) (fun p => Y (p.1, p.2 - Delta))
```

The shift-invariance conditions are *hypotheses*, not axioms. Any concrete temporal frame must discharge them constructively.

7.3 Obligation Persistence Theorem

Theorem 5 (Temporal-lag persistence of ideal obligation)**.** *Let F be a* `DDLPlusFrame` *on $w \times T$ where T is an abelian group, and let* `tub : TemporallyUniformOb F`*. If φ is ideally obligatory at world (v, t_0), then the lag-shifted version $\varphi[\cdot - \Delta]$ is ideally obligatory at $(v, t_0 + \Delta)$.*

Proof sketch. The proof has three parts:

1. *ob part*: `ob_shift` maps $F.\mathrm{ob}(F.\mathrm{pv}\,(v, t_0),\ \varphi)$ to $F.\mathrm{ob}(F.\mathrm{pv}\,(v, t_0{+}\Delta),\ \lambda p.\, \varphi(p_1, p_2{-}\Delta))$, which is exactly `meaningLag phi Delta`.
2. *pv part*: the violation witness (v_w, t_w) is shifted to $(v_w, t_w + \Delta)$ using `pv_shift`.
3. *Negation*: at the shifted witness, `add_sub_cancel_right` reduces the lag-shift evaluation back to the original negation. □

Remark 6. *The "lag" direction is the only direction provable from* `ob_shift` *as stated: the frame does not directly relate ob at two different base times without the content shift. Shifting the reference time forward requires adjusting all content backward to keep the same relative obligations.*

7.4 Causal-Normative Prohibition

We connect the temporal-deontic bridge to PLN's causal heuristics [5]. A *spurious causal candidate* is an element x at time t with extensional but not intensional predictive support:

```
structure SpuriousCausalCandidate
    (profile : PredictiveCausalProfile Domain Time)
    (x : Domain) (t : Time) : Prop where
  extensional    : profile.extensional x t
  not_intensional : not (profile.intensional x t)
```

Theorem 7 (Spurious prohibition)**.** *If a* DTS *obligation structure obliges only intensionally-grounded elements (`ObligesOnlyIntensional`), then no spurious causal candidate can be obligated.*

The proof is a one-line contradiction: `hSpur.not_intensional` contradicts `hGnd.ob_intensional x hOb t`. The contrapositive (`obligatory_not_spurious`) is derived immediately.

8 Abstract Modal Query Encoder

The event-calculus [9, 11] and the governance reasoning layer both define query encoders of the form `Modality -> Referent -> Query`, but they were independently defined. We introduce an abstract structure that unifies both:

```
structure ModalQueryEncoder (Modality Referent Query : Type*) where
  encode : Modality → Referent → Query
```

Event-calculus instance. The event calculus uses three temporal modalities:

```
inductive EventModality where
  | holdsAt      -- fluent holds at time t
  | initiatedAt  -- fluent initiated at t
  | terminatedAt -- fluent terminated at t
```

An `EventQueryEncoder Event Time Query` (with fields `holdsAt`, `initiatedAt`, `terminatedAt`) is equivalent to a `ModalQueryEncoder EventModality (Event x Time) Query`:

```
def eventQueryEncoderToModal (enc : EventQueryEncoder Event Time
    Query) :
    ModalQueryEncoder EventModality (Event x Time) Query where
  encode
    | .holdsAt,      (e, t) => enc.holdsAt e t
    | .initiatedAt,  (e, t) => enc.initiatedAt e t
    | .terminatedAt, (e, t) => enc.terminatedAt e t
```

Theorem 8 (Event-encoder roundtrip)**.** *The maps `eventQueryEncoderToModal` and `modalToEventQueryEncoder` are mutually inverse. Both roundtrips are proven as simp lemmas using `cases modality` and `rfl`.*

Deontic instance. The `DeonticQueryEncoder.modalQuery` field has exactly the shape `DeonticModality -> Eventuality Entity Pred -> Query`, giving:

```
def deonticQueryEncoderToModal (d : DeonticQueryEncoder Entity Pred
     Query) :
    ModalQueryEncoder DeonticModality (Eventuality Entity Pred)
        Query where
  encode := d.modalQuery
```

The embedding is a direct field projection. With this unification, dispatch infrastructure, traversal utilities, and test harnesses written for `ModalQueryEncoder` apply to both the event-calculus and deontic encoders without duplication.

9 GSLT Integration

The governance layer integrates with the GSLT (*Galois-connection Semantics in Logical Form with Types*) categorical framework through the `GovernanceGSLTVertex` module.

A `GovernanceDDLBundle` packages a DDL+ frame together with the governance accessibility data and the CJ theorems:

```
structure GovernanceDDLBundle where
  acc   : ClosedGovAccessibility
  frame : GovFrame { p : Pattern // acc.live p }
```

The `GovernanceForwardFiber` embeds this bundle as a unit-indexed presheaf fiber in the GSLT vertex layer: the single fiber at the unit index contains the governance rewrite rules derived from the CJ theorems. `govNormCycleBundle` provides the concrete instance using `govNormCycleAccessibility`.

10 Gewirth's Principle of Generic Consistency

The Principle of Generic Consistency (PGC) [4] asserts:

> *Act in accord with the generic rights of your recipients as well as of yourself.*

Gewirth's argument proceeds from agency: an agent who acts on purpose must claim the necessary conditions for agency (freedom and well-being, *FWB*) as rights, and by universalizability must grant the same rights to all agents.

The formalization in `Mettapedia/Ethics/` uses the DDL+ embedding developed above:

- `Core.lean`: introduces the agent world type, purpose-directed action predicate `PPA`, and the generic rights type `FWBRight`.
- `GewirthPGC.lean`: packages a `PGCInterpretation` (frame + assignments) and states the main theorem: `PGC_strong_ofAssumptions` proves that any agent satisfying `PGCAssumptions` is ideally obligated to respect the FWB rights of all purposive agents.
- `GewirthBridge.lean`: connects the local `Oi` and `Ocond` operators in `GewirthPGC.lean` to the corresponding `ideal_obl` and `cond_obl` from `DDLPlus.Core`, making the bridge explicit.

The DTS interdefinability theorems (§4) underlie the Gewirth formalization: the equivalences between obligation, permission, and optionality ensure that the logical structure of the PGC proof is sound regardless of which representation is chosen.

11 Related Work

Isabelle/HOL formalization. Benzmüller et al. [2] formalized DDL+ in Isabelle/HOL using the same shallow semantic embedding approach. Our Lean 4 formalization is a port of that work, extended with the governance reasoning layer, temporal bridges, and PLN integration. The Isabelle source (`CJDDLplus.thy`, `ExtendedDDL.thy`) is included in the repository as a reference.

Input/Output logic. Makinson and van der Torre's input/output logic [10] provides an alternative semantics for conditional obligation. The `ob` dyadic relation in DDL+ can be interpreted in their framework, though the present formalization does not develop this connection explicitly.

Event calculus in Lean. The PLN Probabilistic Event Calculus formalized in `PLNProbabilisticEventCalculus.lean` (this repository) follows the Kowalski-Sergot [9] and Shanahan [11] presentations. The `ModalQueryEncoder` of §8 unifies this with the deontic layer.

12 Conclusion

We have presented a Lean 4 formalization of the full governance deontic logic stack: from DDL+ semantic embedding to DTS algebra, Hobbs eventuality reification, constructive reactive accessibility, temporal-deontic bridges, abstract modal query unification, and the Gewirth PGC.

The formalization is:

- **Sorry-free**: 0 uses of `sorry` across all files.
- **Axiom-free**: 0 custom `axiom` declarations; all "conditions" are structure fields discharged by each concrete instance.
- **Compositional**: each layer is independently usable.
- **Grounded**: all semantic conditions trace back to published sources (Carmo & Jones 2002, von Wright 1951, Hobbs 1985, ISO 24617-4).

Future work includes: (i) a full probabilistic evidence integration connecting the DTS strength values to PLN posterior-mean evidence; (ii) automated synthesis of CJ-theorem instantiations for concrete policy domains using the GSLT rewrite engine; and (iii) temporal reasoning about norm revision and contrary-to-duty repair using the event calculus layer.

References

[1] Carlos E. Alchourrón and Eugenio Bulygin. *Normative Systems*. Springer, 1971.

[2] Christoph Benzüller, Ali Farjami, and Xavier Parent. A dyadic deontic logic in HOL. In *Deontic Logic and Normative Systems*, volume 10833 of *Lecture Notes in Artificial Intelligence*, pages 33–49. Springer, 2018.

[3] José Carmo and Andrew J.I. Jones. Deontic logic and contrary-to-duties. In *Handbook of Philosophical Logic*, volume 8, pages 265–343. Kluwer, 2nd edition, 2002.

[4] Alan Gewirth. *Reason and Morality*. University of Chicago Press, 1978.

[5] Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, 2008.

[6] Jerry R. Hobbs. Ontological promiscuity. In *Proceedings of the 23rd Annual Meeting of the Association for Computational Linguistics*, pages 61–69, 1985.

[7] Language resource management — semantic annotation framework (SemAF) — part 4: Semantic roles. ISO Standard 24617-4, 2014.

[8] David Kaplan. Demonstratives. In Joseph Almog, John Perry, and Howard Wettstein, editors, *Themes from Kaplan*, pages 481–563. Oxford University Press, 1989.

[9] Robert Kowalski and Marek Sergot. A logic-based calculus of events. *New Generation Computing*, 4(1):67–95, 1986.

[10] David Makinson and Leendert van der Torre. Input/output logics. In *Journal of Philosophical Logic*, volume 29, pages 383–408, 2000.

[11] Murray Shanahan. The event calculus explained. In *Artificial Intelligence Today*, volume 1600 of *Lecture Notes in Computer Science*, pages 409–430. Springer, 1999.

[12] Georg Henrik von Wright. Deontic logic. *Mind*, 60(237):1–15, 1951.